

PRÄDIKATE

Prädikate sind Aussagen über mathematische Objekte, die wahr oder falsch sein können. «Funktionen» mit booleschen Rückgabewerten: $P, Q(n), R(x, y, z)$. Siehe DMI.

NORMALFORMEN

Normalformen (allgemein, **kanonische Formen**) helfen, das Vergleichsproblem zu lösen (ob zwei Aussagen dieselben sind).

NEGATION

Nicht für alle = Es gibt einen Fall, für den nicht

$P_1 \wedge \dots \wedge P_n = \forall i \in \{1, \dots, n\} (P_i)$

$P_1 \vee \dots \vee P_n = \exists i \in \{1, \dots, n\} (P_i)$

MENGEN

KONSTRUKTION

Grundmenge G , Prädikat $P(x)$. Konstruierte Menge $A = \{x \in G \mid P(x)\}$

OPERATIONEN

Begriff	Bedeutung
Vereinigung	$A \cup B = \{x \mid x \in A \vee x \in B\}$
Schnittmenge	$A \cap B = \{x \mid x \in A \wedge x \in B\}$
Komplement	$\bar{A} = \{x \mid x \notin A\}$
Differenz	$A \setminus B = \{x \in A \mid x \notin B\}$

TUPEL

$A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$

n -Tupel:

$\bigotimes_{i=1}^n A_i = \{(a_1, a_2, \dots, a_n) \mid a_i \in A_i\}$

BEWEISE

Eine Folge von logischen Schlüssen, die zeigen, dass die Aussage aus den gegebenen Voraussetzungen folgt.

Beweistechnik	Anwendungsfall
Konstruktiver Beweis	Liefert explizite «Lösung» / Algorithmus
Widerspruchsbeweis	Geeignet für Unmöglichkeitsaussagen. Z.B. $\sqrt{2} \notin \mathbb{Q}$
Vollständige Induktion	Für Aussagen, die für alle $n \in \mathbb{N}$ gelten.

SPRACHEN

Kontextfrei

CFG
PDA
Palindrome
 $\{0^n 1^n \mid n \geq 0\}$

Turing-erkennbar

Primzahlen
 $\{a^n b^n c^n \mid n \geq 0\}$

Begriff	Bedeutung
Alphabet	Eine nichtleere endliche Menge Σ heisst Alphabet . Die Elemente von Σ heissen Zeichen .
Wort	Eine Zeichenkette der Länge n ist ein n -Tupel in $\Sigma^n = \Sigma \times \dots \times \Sigma$. Ein Element von Σ^n heisst Wort der Länge n . Wörter haben immer endliche Länge.
Leeres Wort	Die Zeichenkette $\varepsilon \in \Sigma^0 = \{\varepsilon\}$ der Länge 0 heisst das leere Wort .

Begriff	Bedeutung
Menge aller Wörter	Leeres Wort ist immer drin $\Sigma^* = \{\varepsilon\} \cup \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \dots = \bigcup_{k=0}^{\infty} \Sigma^k$
Abgekürzte Schreibweisen von Wörtern	$\langle A, u, t, o, S, p, r \rangle = AutoSpr$ $a^3 b^5 = aaabbbbbb$ $b^5 a^3 = bbbbaaaa$
Sprache	Teilmenge $L \subset \Sigma^*$

WORTLÄNGE

Begriff	Bedeutung
Länge des Wortes	$w \in \Sigma^n \Rightarrow w = n$, n ist Länge des Wortes w
Anzahl Zeichen	Sei $w \in \Sigma^*$, $a \in \Sigma$. Dann ist $ w _a$ die Anzahl Zeichen a im Wort w

Beispiel 1 : Wortlänge

$|\varepsilon| = 0$ $|01010|_1 = 2$ $|a^3 b^5| = 8$

$|01010|_0 = 3$ $|01010|_3 = |\text{01291}|^7 = 7|1291| = 7 \cdot 4 = 28$

Beispiel 2 : Sprache

$L = \Sigma^* \subset \Sigma^*$

$L = \emptyset \subset \Sigma^*$, die leere Sprache

$\Sigma = \{0, 1\}$, Sprache aller Binärstrings: $L = \Sigma^*$

$\Sigma = \text{Unicode}$, $J = \{w \in \Sigma^* \mid w \text{ wird vom Java-Compiler akzeptiert}\}$

M eine "Maschine", $L(M) = \{w \in \Sigma^* \mid w \text{ wird von } M \text{ akzeptiert}\}$

Beobachtungen 1

1.1. $|w^n| = n \cdot |w|$

1.2. Zu jeder Maschine M gibt es eine Sprache $L(M)$

DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)

Definition **Tabellenform** **Zustandsdiagramm von A**

$A = (\mathbb{Q}, \Sigma, \delta, q_0, F)$

– Zustände: $\mathbb{Q} = \{q_0, q_1, \dots, q_n\}$

– Alphabet: Σ

– Übergangsfunktion: $\delta : \mathbb{Q} \times \Sigma \rightarrow \mathbb{Q}$

– Startzustand: $q_0 \in \mathbb{Q}$

– Akzeptierzustände: $F \subseteq \mathbb{Q}$

Alle Sprachen sind überabzählbar unendlich

Wichtig: Ein Pfeil für jedes Zeichen in jedem Zustand für validen DEA

WÖRTER EINER REGULÄREN SPRACHE

ÜBERGANGSFUNKTIONEN FÜR WÖRTER

$\delta : \mathbb{Q} \times \Sigma^* \rightarrow \mathbb{Q} : (q, w = a_1, \dots, a_n) \mapsto \delta(\dots \delta(\delta(q, a_1), a_2), \dots, a_n)$

Übergänge ausgehend von q für Zeichen $a_1, \dots, a_n$ nacheinander anwenden.

WORT AKZEPTIEREN

Der DEA $A = (\Sigma, \mathbb{Q}, q_0, \delta, F)$ **akzeptiert** das Wort $w \in \Sigma^*$, wenn er A von Startzustand in einen Akzeptierzustand $\delta(q_0, w) \in F$ überführt.

AKZEPTIERTE SPRACHE, REGULÄRE SPRACHE

Gegeben ein DEA $A = (\Sigma, \mathbb{Q}, q_0, \delta, F)$. Die von A **akzeptierte Sprache** ist

$L(A) = \{w \in \Sigma^* \mid A \text{ akzeptiert } w\} = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}$

Die Sprache $L \subset \Sigma^*$ heisst **regulär**, wenn es einen DEA A gibt mit $L(A) = L$

Beispiel 3 : DEA, der die Sprache $L = \{w \in \Sigma^* \mid w \text{ ist aufsteigend sortiert}\}$ akzeptiert

Start $\rightarrow q_0$

$0, 2$ 1 $0, 2$

θ	1	2
g	g	u
u	u	g

MYHILL-NERODE AUTOMAT

Für ein Wort $w \in \Sigma^*$ setze $L(w) = \{w' \in \Sigma^* \mid ww' \in L\}$. Insbesondere $L(\varepsilon) = L$

Gegeben: reguläre Sprache L über Σ . Rekonstruiere A mit:

- $\mathbb{Q} = \{L(w) \mid w \in \Sigma^*\}$
- $q_0 = L(\varepsilon) = L$
- $F = \{L(w) \in \mathbb{Q} \mid \varepsilon \in L(w)\}$
- $\delta(L(w), a) = L(wa)$

Gleicher Zustand $\Leftrightarrow$ gleiches $L(w)$

Beispiel 4 :

$\Sigma = \{0, 1\}$, $L = \{w \in \Sigma^* \mid |w|_0 \text{ gerade}\}$

Start $\rightarrow q_0$

w	$L(w)$	$\mathbb{Q}$
ε	$L(\varepsilon) = L$	q_0
0	$L(0) = \{w \in \Sigma^* \mid w _0 \text{ ungerade}\}$	q_1
1	$L(1) = L$	q_0
$\vdots$	$\vdots$	$\vdots$

MINIMALAUTOMAT

Ziel: Nicht unterscheidbare Zustände zusammenlegen

Vorgehen:

- 1) Akzeptierzustand $\neq$ Nichtakzeptierzustand
- 2) Iteration: alle unterscheidbaren Paare suchen
- 3) Verbleibende Paare sind ununterscheidbar $\rightarrow$ zusammenlegen

Beispiel 5

Start $\rightarrow q_0$

0 1

	q_0	q_1	q_2	q_3
q_0	$\times$	$\times$	$\times$	$\times$
q_1	$\times$	$\equiv$	$\equiv$	$\times$
q_2	$\times$	$\equiv$	$\equiv$	$\times$
q_3	$\times$	$\times$	$\times$	$\equiv$

Algorithmus

- 1) $q \in F, q' \notin F$
- 2) Unterscheidbar nach Übergang
- 3) Verbleibende Paare sind ununterscheidbar $\rightarrow$ zusammenlegen

$\Rightarrow q_1$ und q_2 sind nicht unterscheidbar

Start $\rightarrow q_0$

$0, 1$ 1

0

q_1, q_2

q_3

PUMPING LEMMA

ist L eine reguläre Sprache, dann gibt es $N \in \mathbb{N}$, die pumping length so, dass jedes Wort $w \in L$ mit $|w| \geq N$ in drei Teile $w = xyz$ zerlegt werden kann mit:

- 1) $|xy| \leq N$
- 2) $|y| > 0$
- 3) $xy^k z \in L \forall k \in \mathbb{N}$

Oder: genügend lange Wörter ($|w| \geq N$) einer regulären Sprache ($w \in L$) können alle in einem Anfangsstück der Länge N ($|xy| \leq N$) aufgepumpt werden ($xy^k z \in L$).

Was zeichnet reguläre Sprachen aus?

- Reguläre Ausdrücke
- Lange Wörter: * kommt im regulären Ausdruck vor
- Blöcke mit * können beliebig oft wiederholt werden
- $\Rightarrow$ Wörter können «aufgepumpt» werden

BEWEIS

Behauptung: Die Sprache $L \subset \Sigma^*$ ist nicht regulär.

Beweis (Widerspruch):

- 1) Annahme: L ist regulär
- 2) Gemäss Pumping Lemma gibt es die Pumping Length N
 - Es darf keine Annahme über die konkrete Grösse von N gemacht werden!
- 3) Wähle ein Wort $w \in L$ mit $|w| \geq N$
 - Die Definition **muss** N verwenden!
- 4) Aufteilung des Wortes gemäss Pumping Lemma
 - $w = xyz$, $|xy| \leq N$, $|y| > 0$
- 5) Auswirkung des Pumpens
 - $\Rightarrow xy^k z \notin L$ für mindestens ein $k \in \mathbb{N}$ (mit Begründung!)
- 6) Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

Beispiel 6 : $L = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

1) Annahme: L ist regulär

2) $\exists N \in \mathbb{N}$, Pumping Length

3) $w = 0^N 1^N$

4) Unterteilung $w = xyz$

TODO: reduce confusion

5) Pumpen: nur die Anzahl der 1 wird erhöht, Anzahl 0 bleibt

6) $xy^k z \notin L$ für $k \neq 1$, im Widerspruch zum Pumping Lemma

NICHTDETERMINISTISCHE ENDLICHE AUTOMATEN (NEA)

Definition **Akzeptieren**

$A = (\mathbb{Q}, \Sigma, \delta, q_0, F)$

– Endliche Menge von Zuständen: $\mathbb{Q}$

– Alphabet: Σ

Ein NEA A **akzeptiert** das Wort $w \in \Sigma^*$, wenn es eine **Wahl** von Übergängen gibt, derart, dass das Wort

– Übergangsfunktion: $\delta : \mathbb{Q} \times \Sigma \rightarrow P(\mathbb{Q})$

– Startzustand: $q_0 \in \mathbb{Q}$

– Akzeptierzustände: $F \subseteq \mathbb{Q}$

$F \subset \mathbb{Q}$ $0, 1$

w den Automaten in einen Akzeptierzustand überführt.

Faustregeln

- Nur genau diejenigen Pfeile einzeichnen, die man zum Akzeptieren braucht.
- Erlaube Alternativen

Beispiel 7 : $L = \{w \in \Sigma^* \mid w \text{ endet mit zwei } 0\}$

DEA-Lösung NEA-Lösung

Start $\rightarrow q_0$

1 0, 1 0

q_0 q_1 q_2 q_3

UMGANG MIT MEHRDEUTIGEN ÜBERGÄNGEN

a-Übergang von q_1 aus nicht eindeutig

Welchen Übergang soll man nehmen?

– Beide Möglichkeiten probieren und akzeptieren, falls eine der Möglichkeiten auf einen Akzeptierzustand führt.

– Zufallsgenerator (probabilistischer endlicher Automat, PEa)

THOMPSON-NEA

– Zustand = Menge der möglichen Zustände

– Akzeptieren = Ob NEA im Zustand sein könnte

Start $\rightarrow \{q_0\}$

a, b

$\emptyset$ $\{q_1\}$ $\{q_2\}$

Transformation NEA $\rightarrow$ DEA

Gegeben $\delta : \mathbb{Q} \times \Sigma \rightarrow P(\mathbb{Q})$ eines NEA. Übergangsfunktion für Mengen $M \subset \mathbb{Q}$

$\delta' : P(\mathbb{Q}) \times \Sigma \rightarrow P(\mathbb{Q}) : (M, a) \mapsto \delta'(M, a) = \bigcup_{q \in M} \delta(q, a)$

Satz

Ein NEA A kann in einen DEA A' umgewandelt werden mit

- $\mathbb{Q}' = P(\mathbb{Q})$
- $\Sigma' = \Sigma$
- δ' wie oben definiert
- $q'_0 = \{q_0\}$
- $F' = \{M \in P(\mathbb{Q}) \mid F \cap M \neq \emptyset\}$

Implementation

Thompson-NEA:

- Zustand $M \subset \mathbb{Q}$ realisiert durch Markierung der Zustände in M
- δ' realisiert durch Verschieben von Markierungen
- Akzeptieren: mindestens ein Akzeptierzustand markiert

NEA _ε	
Begriff	Bedeutung
ε-Übergang	Kann ohne Verarbeitung eines Zeichens genommen werden.
$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow P(Q)$	
NEA _ε	NEA mit ε-Übergängen

Einen NEA_ε kann man immer in einen NEA umwandeln.
Beweis:

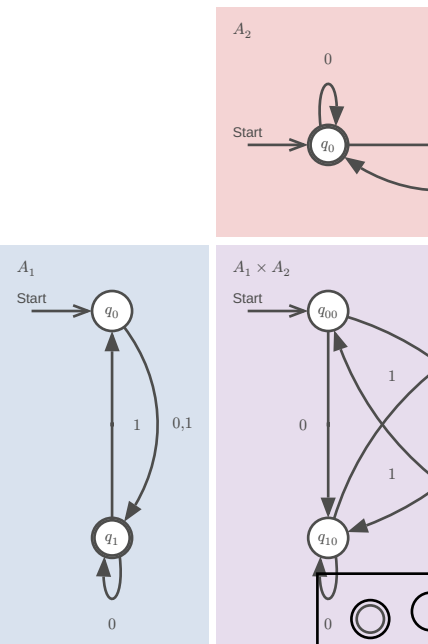
- 1) $E(q)$ = Menge der von q aus mit ε -Übergängen erreichbaren Zustände
- 2) $E(M) = \bigcup_{q \in M} E(q)$
- 3) δ ersetzen durch $\delta' : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow P(Q) : (q, a) \mapsto E(\delta(q, a))$

MENGENOPERATIONEN

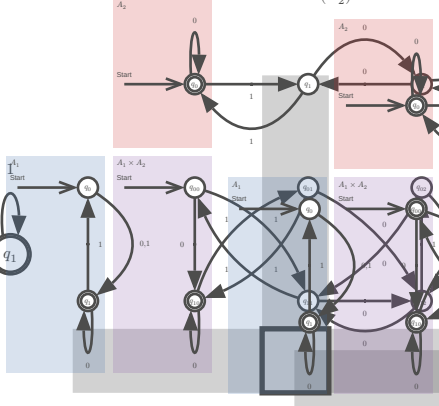
Lassen reguläre Sprachen regulär sein.

Produktautomat: $L_1 = L(A_1)$

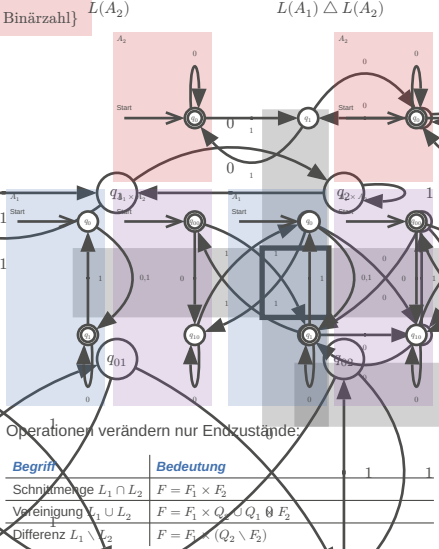
$\Sigma^* \mid |w|_1 \text{ ungerade} \cap \{w \in \Sigma^* \mid w \text{ ist eine durch drei teilbare Binärzahl}\}$



Schnittmenge $L(A_1) \cap L(A_2)$ Vereinigungsmenge $L(A_1) \cup L(A_2)$



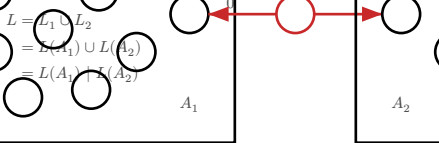
Differenzmenge $L(A_1) \setminus L(A_2)$ Symmetrische Differenz $L(A_1) \Delta L(A_2)$



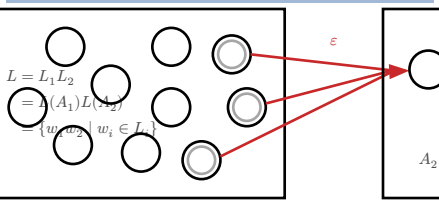
Begriff	Bedeutung
Schnittmenge $L_1 \cap L_2$	$F = F_1 \times F_2$
Vereinigung $L_1 \cup L_2$	$F = F_1 \cup Q_2 \cup F_2$
Differenz $L_1 \setminus L_2$	$F = F_1 \times (Q_2 \setminus F_2)$

REGULÄRE OPERATIONEN

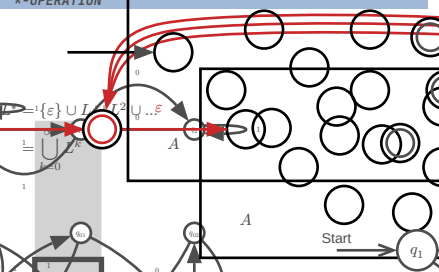
WICHTIG: ε-Übergänge immer hinzufügen!



VERKETTUNG



*-OPERATION



⇒ die Klasse der regulären Sprachen ist abgeschlossen unter regulären Operationen

REGULÄRE AUSDRÜCKE

Formeln, die Zeichenketten beschreiben.

- 1) Buchstaben stehen für sich selbst, mit Ausnahme der Metazeichen ($\backslash$ (escape character))
- 2) Verkettung: Zeichen und Formeln hintereinanderschreiben
- 3) $\cdot$ = ein beliebiges Zeichen, Σ
- 4) $|$: Alternative, $a|A = a \text{ oder } A$
- 5) $*$: Wiederholung, beliebige viele
- 6) $()$: Gruppierung
- 7) $[]$ Zeichenklassen: $[abc] = a|b|c$, $[^abc] = \text{alles ausser } a, b, \text{ oder } c$
- Erweiterungen / Dialekte
- 1) $\{n, m\}$: zwischen n und m Wiederholungen
- 2) $+$: mindestens eines, $\{1, \}$
- 3) $?$: optional, $\{0, 1\}$
- 4) Symbole für Zeichenklassen: $\backslash d$ Ziffern, $\backslash s$ whitespace, $[:space:]$ (upper :)

VERALLGEMEINERTER NEA (VNEA)

Regulärer Ausdruck

Reguläre Operationen

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Primitive reguläre Ausdrücke

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

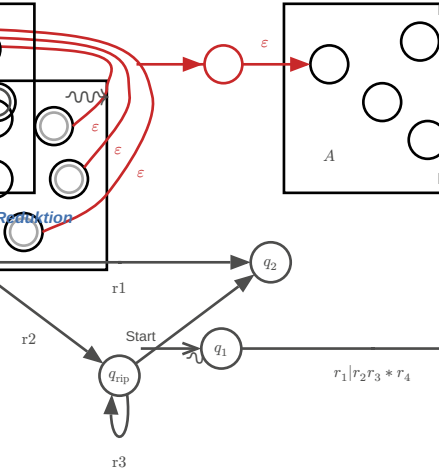
Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

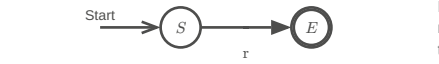
Reguläre Ausdrücke für Wörter mit Länge ≤ 1

Reguläre Ausdrücke für Wörter mit Länge ≤ 1



Regulärer Ausdruck

Nach Entfernen aller Zwischenzustände $q_{rip} \in Q$ bleibt ein regulärer Ausdruck r von A



⇒ jede reguläre Sprache lässt sich mit einem regulären Ausdruck beschreiben $\{L(A) \mid A \text{ ein DEA}\} = \{L(r) \mid r \text{ ein regulärer Ausdruck}\}$

Interessantes projekt (DEA Lexer DSL): [Ragel](#)

TESTSTRATEGIE

Messung	Folgerung
Für $n = 1, 2, 3, \dots$	– Laufzeit $O(2^n)$: NEA-Implementierung
– Regulären Ausdruck erzeugen $r = a^? a^? a^? a^? \dots a^? aaaa \dots a$	– Laufzeit $O(n)$: DEA-Implementierung
– Laufzeit für Akzeptieren von a^n durch r messen	

KONTEXTFREIE SPRACHEN

Begriff	Bedeutung
CFL	Context Free Language
CFG	Context Free Grammar
PDA	Pushdown Automata

KONTEXTFREIE GRAMMATIK UND SPRACHE

Kontextfreie Grammatik: **Parse Tree**

$G = (V, \Sigma, R, S)$

$L = \{w \in \{0, 1\}^* \mid |w|_0 = |w|_1\}$

mit der Grammatik

$S \rightarrow 0S1 \mid 1S0 \mid SS \mid \varepsilon$

$S \rightarrow aA \mid C$

$A \rightarrow a$

$A \rightarrow aAb$

$C \rightarrow bA$

$S \rightarrow aC \rightarrow aA$

$S \rightarrow bC \rightarrow bA$

$S \rightarrow C \rightarrow ?$

CHOMSKY-NORMALFORM (CNF)

Eine CFG ist in **Chomsky-Normalform**, wenn S auf der rechten Seite nicht vorkommt und jede Regel von der Form $A \rightarrow BC$ oder $A \rightarrow a$ ist, zusätzlich ist die Regel $S \rightarrow \varepsilon$ erlaubt.

Umwandlung

Umwandlung

Umwandlung

Umwandlung

Umwandlung

Umwandlung

Umwandlung

Erzeugte Sprache: Die Menge der Wörter, die von einer kontextfreien Grammatik G aus der Startvariablen abgeleitet werden können, wird mit $L(G)$ bezeichnet und heißt die von G erzeugte Sprache.

Kontextfreie Sprache: Eine Sprache L heißt kontextfrei, wenn es eine kontextfreie Grammatik gibt, die die Sprache $L = L(G)$ erzeugt.

Beispiel 8 : $L = \{0^n 1^n \mid n \geq 0\}$

Grammatik	$L =$	Wörter, die erzeugt werden müssen
$\{0^n 1^n \mid n \geq 0\}$		
1) Variablen: $V = \{S\}$		ε
2) Terminalsymbole: $\Sigma = \{0, 1\}$		$01 = 0\varepsilon 1$
3) Regeln: $R = \{S \rightarrow \varepsilon 0S1\}$		$0011 = 0011$
4) Startvariable: S		

Es ist nicht garantiert, dass es für die Ableitung eines Wortes nur einen einzigen Syntaxbaum gibt. Man sagt, die Grammatik ist **nicht eindeutig**, wenn sie mehrere Syntaxbäume für die gleichen Wörter erlaubt.

REGULÄRE OPERATIONEN

Die Klasse der kontextfreien Sprachen ist abgeschlossen unter regulären Operationen.

Reguläre Sprachen sind kontextfrei.

Grammatik für reguläre Operationen

Seien L_1 und L_2 kontextfreie Sprachen mit Grammatiken $G_i = (V_i, \Sigma, R_i, S_i)$. Wir nehmen an, dass die Variablen- und Regelmengen disjunkt sind, also $V_1 \cap V_2 = \emptyset \wedge R_1 \cap R_2 = \emptyset$. Zudem sei S_0 eine Variable, die nicht in $V_1 \cup V_2$ enthalten ist. Dann gilt:

Alternative $L_1 \mid L_2$ ist kontextfrei mit der Grammatik

$G = (V_1 \cup V_2 \cup \{S_0\}, \Sigma, R = R_1 \cup R_2 \cup \{S_0 \rightarrow S_1 \mid S_2\}, S_0)$

Verkettung $L_1 L_2$ ist kontextfrei mit der Grammatik

$G = (V_1 \cup V_2 \cup \{S_0\}, \Sigma, R = R_1 \cup R_2 \cup \{S_0 \rightarrow S_1 S_2\}, S_0)$

-Operation L_1^ ist kontextfrei mit der Grammatik

$G = (V_1 \cup \{S_0\}, \Sigma, R = R_1 \cup \{S_0 \rightarrow S_0 S_1, S_0 \rightarrow \varepsilon\}, S_0)$

KONTEXTFREI

Regeln ohne Kontext

Es spielt keine Rolle, in welchem Kontext das Zeichen A vorkommt, die Regel kann immer angewendet werden

$S \rightarrow aAb \rightarrow aab$

$S \rightarrow aC \rightarrow aA$

$S \rightarrow bC \rightarrow bA$

$S \rightarrow C \rightarrow ?$

Regeln mit Kontext

Je nach **Kontext** kann eine Regel nicht unbedingt angewendet werden

$S \rightarrow C \mid aC \mid bC$

$aC \rightarrow A$

$bC \rightarrow B$

$S \rightarrow C \rightarrow ?$

Umwandlung

Umwandlung

- Seite 3

Wiederverwendete Variable

$|w| \geq N$ gross genug $\Rightarrow$ Variablen werden im Parse Tree wiederverwendet Die «unterste» wiederverwendete Variable A erzeugt zwei Wörter:

$A \Rightarrow vxy$
 $A \Rightarrow x$

Pumpen

$A \Rightarrow vxy$ anstelle des «untersten» $A \Rightarrow x$ verwenden

TODO:
diagram

UNENDLICH

Begriff	Bedeutung
Endlich	Eine Menge A heisst endlich , wenn jede Injektion $A \rightarrow A$ auch eine Bijektion ist
Abzählbar unendlich	Eine Menge A heisst abzählbar unendlich , wenn es eine Bijektion $\mathbb{N} \rightarrow A$ gibt also $A \simeq \mathbb{N}$. Bsp: $\mathbb{R}, \mathbb{Z}, \mathbb{Q}, \Sigma^*$, Menge DEAs/NEAs/PDAs/CFGs
Überabzählbar unendlich	Eine Menge A heisst überabzählbar unendlich , wenn sie nicht abzählbar unendlich ist. Bsp: $\mathbb{C}, P(\mathbb{N})$, Menge aller Sprachen $P(\Sigma^*)$
Gleich mächtig	Mengen A und B heissen gleich mächtig , $A \simeq B$, wenn es eine Bijektion $A \rightarrow B$ gibt
Unendlich	Eine Menge A heisst unendlich , wenn sie x, R gleich mächtig wie eine Teilmenge ist

A, B, A_k abzählbar unendlich, $k \in \mathbb{N}$:

- $A \cup B, \bigcup_k A_k$ abzählbar unendlich
- $A \times B$ abzählbar unendlich
- Abzählbar unendlich: $\mathbb{N}, \mathbb{Z}, \mathbb{Q}$
- $P(A)$ überabzählbar unendlich
- $\mathbb{R}$ überabzählbar unendlich

TURING-MASCHINE (TM)

Merhere Stacks (Speicherzellen) = RAM (Band).

Jeder DEA oder NEA kann damit emuliert werden.

- Der Speicher ist unbegrenzt
- In jeder Zelle genau ein Zeichen
- Es ist immer nur eine Speicherzelle einsehbar
- Der Inhalt der aktuellen Zelle kann beliebig verändert werden
- Bewegung immer nur eine Zelle nach links oder rechts
- Kein weiterer Speicher (nur die Zustände eines endlichen Automaten und das eine Zeichen)

Begriff	Bedeutung
Record	Speichereinheit fester grösse
Bandalphabet	Alphabet Γ , welches aus Speicherzellen (Stacks) besteht
Schreib-/Lesekopf	Aktuelle Schreib-/Leseposition

Definition

(deterministische) Turing-Maschine

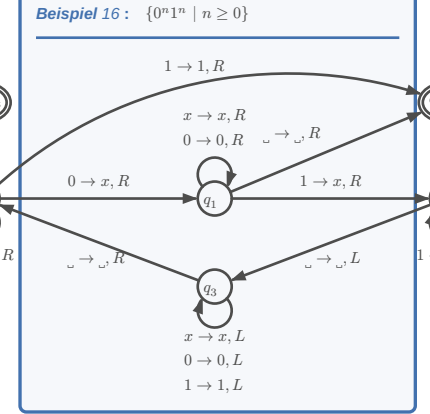
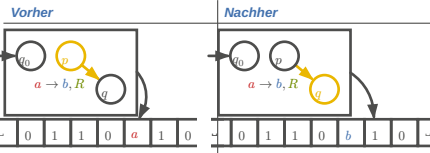
$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$

- Q : Zustände
- Σ : Alphabet
- Γ : Bandalphabet, $\subseteq \Gamma \setminus \Sigma$
- δ : $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$
- $q_0 \in Q$: Startzustand
- $q_{\text{accept}} \in Q$: Akzeptierzustand
- $q_{\text{reject}} \in Q$: Ablehnzustand, $q_{\text{accept}} \neq q_{\text{reject}}$

ARBEITSWEISE
Programmablauf

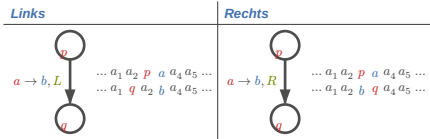
- Input-Wort w auf Band
- Schreib- / Lesekopf positioniert auf 1. Zeichen
- Maschine starten, $t(w)$ Einzelschritte ausführen
- Maschine hält in q_{accept} oder q_{reject} : Wort w akzeptiert / Verworfen

Laufzeit: $t(w), t(n) = \max\{t(w) \mid |w| \leq n\}$

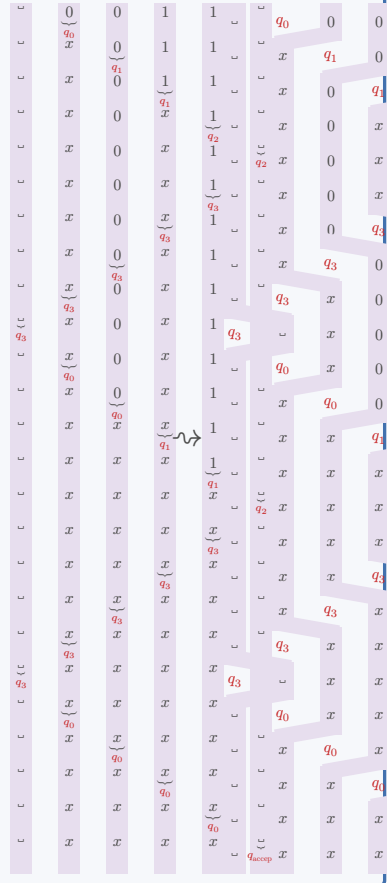


PROTOKOLLIERUNG

Der Gang der Berechnung mit einer TM kann dokumentiert werden, indem der Bandinhalt nach jedem einzelnen Verarbeitungsschritt protokolliert wird.



Beispiel 17: $L = \{0^n 1^n \mid n \geq 0\}, w = 0011$



VARIANTEN

NICHTDETERMINISTISCHE TM

Übergangsfunktion

Bei jedem Übergang maximal N verschiedene Möglichkeiten: $\delta : Q \times \Gamma \rightarrow P(Q \times \Gamma \times \{L, R\})$
 $\Rightarrow$ Mehrere mögliche Berechnungswege

Wort akzeptieren

$w \in L(M)$ wenn es einen Berechnungsweg gibt, der zu q_{accept} führt. (auch wenn es Wege gibt, die auf q_{reject} führen!)

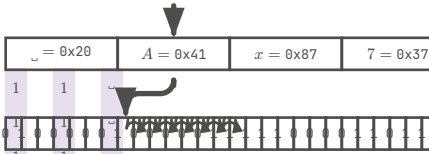
Simulationsidee

Alle Berechnungswege durchführen, maximal $N^{t(n)}$

Simulierbar in $2^{O(t(n))}$

VERSCHIEDENE BANDALPHABETE

Jedes andere Alphabet als $\Sigma = \{0, 1\}$ kann binär codiert werden.



Simulierbar in Zeit $O(t(n))$

MEHRBANDMASCHINE

TODO:

buch ..250.., slides 14

Simulierbar in Zeit $O(t(n))$

MEHRBANDMASCHINE

Um eine Mehrbandmaschine mit n Bändern auf einer einfacheren Maschine zu simulieren, kann die unabhängige Bewegung der Schreib-/Leseköpfe mithilfe eines $2n$ -spurigen Bandes (Daten + Zeiger für Schreib-/Lesekopf) nachgebildet werden.

TODO:

buch ..250..

Simulierbar in Zeit $O(t(n)2)$

Harvard- und von Neumann-Architektur

Die AVR-Mikrokontroller-Familie von Atmel verwendet intern drei unabhängige Datenpfade:
Flash-Speicher, der den Programmcode enthält
Statisches RAM, welches zur Laufzeit veränderliche Daten speichert
Peripheriegeräte.

Die von Neumann-Architektur vereinheitlicht Daten- und Programmspeicher.

EINSEITIG UNENDLICHES BAND

Beidseitig unendliches Band kann durch ein Alphabet doppelter Wortbreite realisiert werden, somit äquivalent.

TODO:

diagram (buch 235)

TURING-ERKENNBARE SPRACHEN

Eine TM erkennt das Wort $w \in \Sigma^*$, wenn die Maschine auf dem Input w im Zustand q_{accept} anhält.

Sind L_1 und L_2 Turing-erkennbare Sprachen, dann sind auch der Durchschnitt $L_1 \cap L_2$ und die Vereinigungsmenge $L_1 \cup L_2$ Turing-erkennbar.

Es gibt überabzählbare viele Sprachen $L \subset \Sigma^*$, die nicht Turing-erkennbar sind.

AUFZÄHLER

Aufzähler: TM mit einem Drucker, mit dem Wörter ausgedruckt werden können.

L ist **rekursiv aufzählbar** wenn es einen Aufzähler gibt, der alle Wörter aufzählen kann. L ist dann auch Turing-erkennbar.

ENTSCHEIDER UND ENTSCHEIDBARE SPRACHEN

Deterministisch: Eine TM M heisst **Entscheider**, wenn sie auf jedem Input w anhält.

Nichtdeterministisch: Eine nichtdeterministische TM M heisst **Entscheider**, wenn jede Berechnungsgeschichte terminiert.

Turing-erkennbare Sprache: L heisst **Turing-erkennbar**, wenn es eine TM M gibt mit $L = L(M)$.

Turing-entscheidbare Sprache: L heisst **Turing-entscheidbar**, wenn es einen Entscheider M gibt mit $L = L(M)$.

TODO:

nicht entscheidbare Probleme

ENTSCHEIDBARKEIT

«Die sich mit immer neuen Superlativen gegenseitig überbietenden Ankündigungen der IT-Industrie könnten den Eindruck erwecken, dass mit geeignet viel Rechenleistung und Speicherplatz kein Informatikproblem einer Lösung widerstehen könnte.»

— Prof. Dr. Andreas Müller, [1, p. 267]

> based

TODO:

(Zehntes hilbertsches Problem). Man gebe ein Verfahren an, welches für eine beliebige diophantische Gleichung entscheidet, ob sie lösbar ist.